

FabriQ: An End-to-End Fabric Defect Detection System Using YOLOv8 and a Web Dashboard

Technical Report (draft for conference / journal paper). **Authors:** [Your Name(s)]. **Affiliation:** [Your Institution]. **Date:** May 2026. **Version:** 1.0.

Abstract

Fabric defect detection is critical for quality assurance in textile manufacturing. Commercial inspection systems are often costly and poorly suited to small and medium enterprises (SMEs). This report describes **FabriQ**, an integrated software stack that combines **YOLOv8-based object detection**, a **Flask REST API**, a **React (Vite) dashboard**, **Firebase (Firestore + Storage)** for persistence, and **live camera streaming** with server-side inference suitable for laptop prototyping through edge devices such as NVIDIA Jetson Nano with industrial or Arducam sensors. The system supports image and video upload, frame-wise annotated video output, real-time MJPEG streams with bounding boxes, and analytics derived from stored defect records. Quantitative benchmarks should be inserted after formal evaluation on a held-out test set.

Keywords: fabric defect detection, YOLOv8, computer vision, Flask, React, Firebase, MLOps, edge deployment, textile industry.

1. Introduction

Textile production involves continuous material movement on rollers, making automated visual inspection valuable for reducing waste and maintaining grade consistency. Deep learning detectors such as YOLO variants offer a practical balance between accuracy and inference latency.

- Standardized pipeline from messy folders to YOLO-compatible datasets.
- Training scripts for classification and detection (including pseudo-annotations where box labels are scarce).
- Web application: detection UI, dashboard metrics, analytics, exportable reports.
- Firebase for defect persistence and optional annotated images.
- Live inspection via backend camera capture and streamed annotated frames.

2. Related Work and Background

Fabric defect detection spans handcrafted features, classical ML, CNNs, and modern detectors/segmentation. **FabriQ** emphasizes **operational completeness** (API + UI + storage + streaming + CI) and deployment to edge cameras rather than a novel backbone architecture.

3. Problem Statement

- **Inputs:** Images/video of fabric; optional live camera on laptop, workstation, or Jetson.
- **Outputs:** Bounding boxes and class labels; annotated previews; stored records for analytics.
- **Constraints:** Low cost, simple UX, swap weights via configuration; edge limits on model size and cameras (USB/CSI).

4. Dataset and Preparation

4.1 Organization

organize_dataset.py maps folder/filename cues to a **20-class** taxonomy, fuzzy term mapping, standardized filenames, and **80/20 train/validation** split under `FabriQ_Final_Dataset`.

4.2 Detection Dataset

create_pseudo_annotations.py can bootstrap full-image YOLO boxes; replace with real annotations over time.

4.3 Application Schema

Backend inference classes: **hole**, **objects**, **oil_spot**, **thread_error**. Firestore normalizes to hole / stain / broken_thread / misweave. **CLASSES** in `backend/app.py` must match **YOLO class order** in weights.

5. Model Training

Ultralytics YOLOv8 via `train_yolo_detection.py`; CUDA when available, CPU with reduced batch/epochs. Address NumPy/OpenCV/PyTorch alignment, corrupt JPEG cleanup, and PyTorch 2.6 weights loading where needed.

Evaluation: Report precision, recall, mAP@0.5, mAP@0.5:0.95, FPS, hardware, resolution, confidence threshold.

6. System Architecture

Frontend React, Vite, Material UI UX, charts, JWT client
Backend Flask, OpenCV, Ultralytics Inference, uploads, live MJPEG
Persistence Firebase Firestore/Storage Defects, optional images
Ops Docker Compose, GitHub Actions Packaging and CI

Upload: POST `/api/detection/analyze` — JSON defects, annotated image base64, annotated video URL for uploads.

Live: POST `/api/live/start` — `cv2.VideoCapture(index)`; GET `/api/live/stream` — MJPEG with boxes; GET `/api/live/latest` — latest frame defect metadata.

Model path priority: `FABRIQ_MODEL_PATH` → relative runs/detect weights → absolute fallback → `yolov8n.pt`. **Camera:** `FABRIQ_CAMERA_INDEX` or UI-selected index.

7. Frontend Application

- Login; Dashboard from Firestore; Detection upload + live stream + camera selector;
- Analytics and Reports; Firestore writes after inference (Storage fallback tolerant).

8. Deployment

- Backend: `venv`, `pip install -r backend/requirements.txt`, `python backend/app.py`.
- Frontend: `npm install`, `npm run dev`, set `VITE_API_URL`.

- Jetson: ARM builds, camera index, consider TensorRT for latency.

9. MLOps

.gitignore for weights/runs/datasets; GitHub Actions CI; MLOPS_GUIDE.md; tag releases with dataset/model versions.

10. Limitations and Future Work

- Replace pseudo full-image boxes with precise labels.
- Batch or backend-write if live Firestore logging grows.
- Explainability, RBAC, TensorRT/GStreamer as extensions.

11. Conclusion

FabriQ provides a **complete modular stack** for fabric defect detection suitable for SMEs and edge rollout, ready for a full paper once empirical metrics are added.

References (replace with formal citations)

- Ultralytics YOLOv8 — <https://docs.ultralytics.com>
- Flask — <https://flask.palletsprojects.com>
- Firebase — <https://firebase.google.com/docs>

Appendix A: Key Files

organize_dataset.py Dataset structuring
train_yolo_detection.py Detection training
create_pseudo_annotations.py Bootstrap labels
backend/app.py API and live stream
frontend/ Dashboard
fabriq_detection_data.yaml YOLO data config
MLOPS_GUIDE.md Operations

Appendix B: Figures for Camera-Ready Paper

- Architecture diagram
- Sample detections with bounding boxes
- Training/validation curves
- Dashboard screenshots